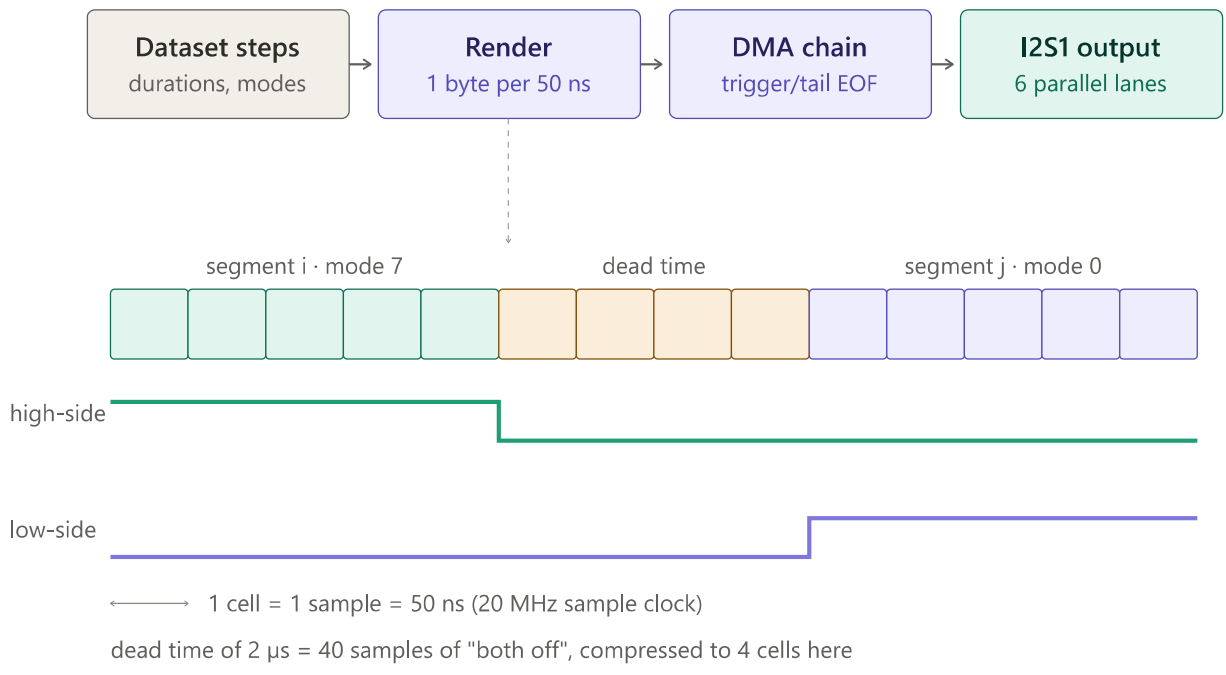
DMA signal engine: timing, analog reading, and the "age vs latency" question

Study of `signal_engine_dma.cpp`, `signal_controller.cpp` and `helper_analog.cpp` · default 60 µs pattern · generated 2026-07-04

1 · Signal generation: timing comes from hardware, not code

The DMA engine does not toggle GPIOs from software. The whole signal cycle is pre-computed as a byte stream by `signal_render_bitstream()` and the I2S1 peripheral clocks it out in 8-bit parallel mode at **20 MHz — one byte every 50 ns**. Each byte carries the state of all six gate pins (one bit per lane). Dead time is not a runtime delay: it is 40 samples of "both switches off" baked into the stream (2 µs at the default dead time). Once the buffer plays, signal timing needs zero CPU involvement.

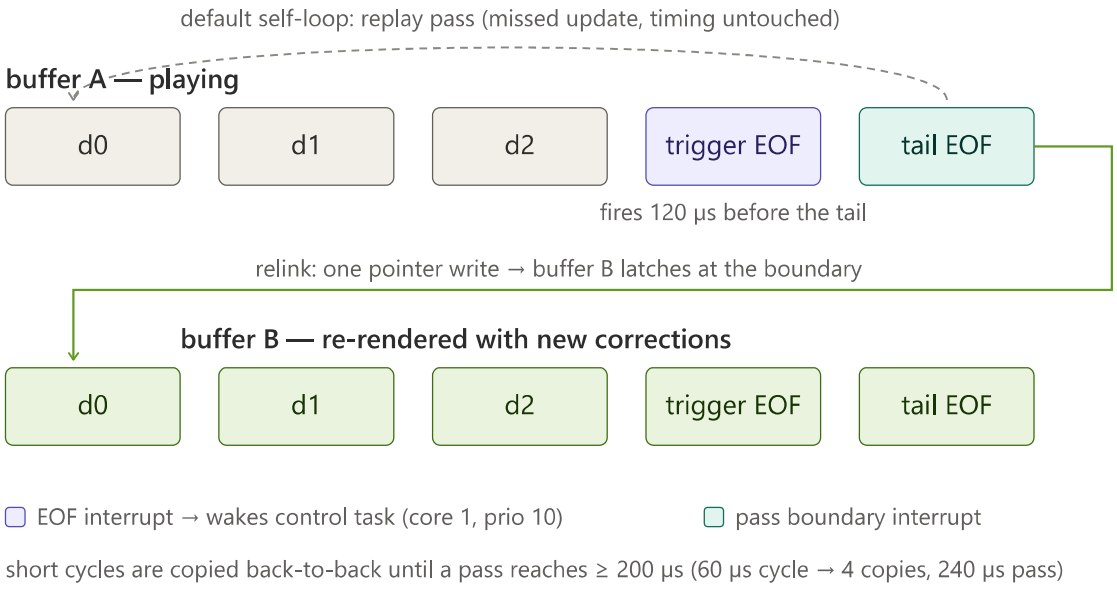


From dataset to hardware: the render step expands each segment into 50 ns samples, inserting "both off" dead-time samples at every transition. The I2S peripheral replays them with hardware-clock precision.

The rendered buffer is wrapped in a descriptor chain (`chain_build()`) with two special descriptors:

- **Trigger EOF** — placed `DMA_CONTROL_LEAD_US = 120 µs` of playback before the pass boundary. Its interrupt wakes the control task (Core 1, priority 10).
- **Tail EOF** — at the pass boundary. It counts passes and, by default, *links back to its own chain head* (self-loop).

> The self-loop is the key safety property: if the control task does not finish inside the 120 µs window, the hardware simply replays the same pass. That is a *Missed Control Update* — the correction is skipped, but the power signal timing is completely untouched.



Double buffering with the descriptor chain. The commit is a single atomic pointer write on the tail descriptor, so the swap can never tear the waveform.

2 · Reading the analog signals

The reading side runs on Core 0, fully decoupled from the signal engine:

1. The ADC samples AN3/AN5/AN6 round-robin at 250 kHz aggregate — one triple every 12 µs, free-running, with *no phase relationship* to the signal edges.
2. Samples accumulate in the ADC DMA driver into frames of 16 triples (48 samples = **192 µs of signal time per frame**). The driver store holds up to 4 frames; `flush_pool = 1` prefers

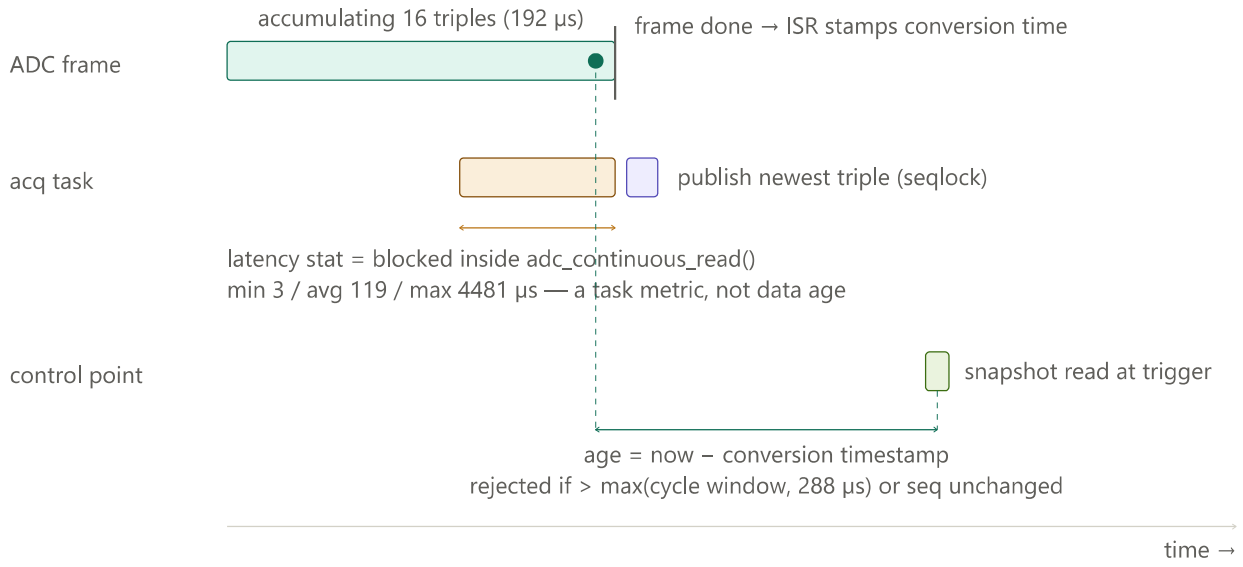
dropping old data over keeping a backlog.

3. When a frame completes, the `on_conv_done` ISR captures a timestamp — this is what makes the age of each sample measurable later.
4. The acquisition task reads the frame, **backdates every sample** to its true conversion time (frame timestamp minus position × sample period), assembles complete triples, and publishes **only the newest one** through a seqlock. Older triples in the same batch are discarded on purpose — the control point only ever wants "the latest".

2.1 · The age of an element — and why it is not the latency statistic

The status values below are *two different measurements taken at two different places*, and only one of them ever touches the values the controller uses:

Statistic	What it actually measures	Where in code
`ADC latency min 3 µs`	A completed frame was already waiting in the driver store — the read call returned immediately.	Stopwatch around `adc_continuous_read()` in `analog_continuous_step()` — **how long the acquisition task sat blocked waiting for a frame**. A Core-0 health metric. Never attached to a sample, never gates a value.
`ADC latency avg 119 µs`	The task usually arrives mid-frame and waits the remainder of the 192 µs accumulation (~half a frame ≈ 96 µs + wakeup overhead). This is the *healthy* signature.	
`ADC latency max 4481 µs`	One call took 4.5 ms of wall clock. Frames complete every 192 µs, so this means the *task itself did not get CPU time* (Core 0 busy, typically a BLE burst) — not that the ADC stopped.	
`age`	now − conversion timestamp of the snapshot, computed at the moment of *use* in `analog_read_control_snapshot()`. The snapshot is **rejected** (recorded miss) if age exceeds max(cycle window, 288 µs pipeline floor) or if its sequence number has not advanced since the last use.	`helper_analog.cpp`, control-point read path



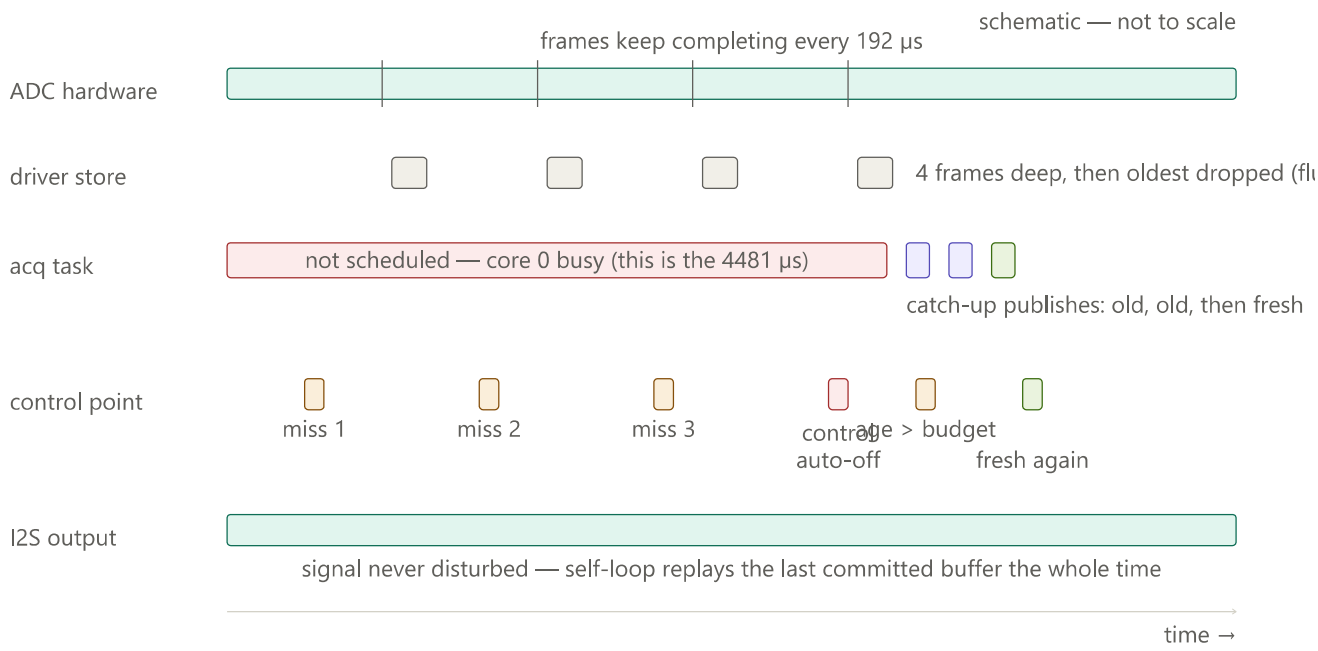
Latency is measured on the reader task (orange bracket); age is measured on the data itself, from conversion time to consumption (green bracket). They overlap in time but are independent quantities.

Q: Am I using a value with the max latency?

No — twice over. First, latency is not a property of any value; it is the reader task's blocking time. Second, even when a 4.5 ms stall makes the eventually-published data old, the age gate rejects it at the control point. The consequence of a stall is **missed correction updates, never actuation on old data**. Side effect worth knowing: 3 consecutive misses auto-disable the controller (`signal_control_update_corrections()`), so if control has ever dropped to OFF by itself, these max-latency events are the likely cause.

Q: Can some newer element be used instead?

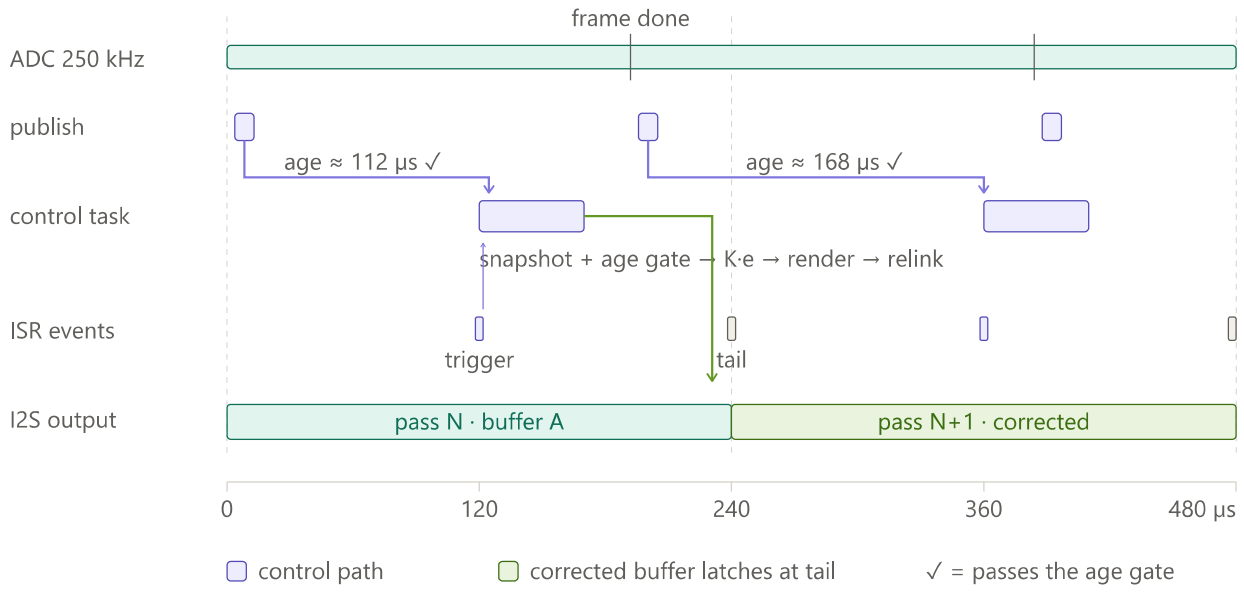
There is no newer *published* element to pick — the pipeline already discards everything except the newest triple per read. During a stall, newer samples do exist, but as raw unread bytes inside the ADC DMA store. The remedy is making the reader run sooner (Core-0 scheduling, smaller frames), not selecting a different element.



The max-latency event end to end: the ADC never stops, data queues unread, the age gate holds the line, and the only casualty is the correction update rate.

3 · How generation and reading work together

The contract: a measurement taken during pass N is consumed at the trigger (120 µs before N ends), corrections are rendered and relinked before the tail, and pass N+1 plays them. The control cadence is one update per pass (240 µs for the default pattern), which sits comfortably above the 192 µs ADC frame period — so the DMA engine almost always finds a fresh snapshot waiting (unlike the CPU engine, whose 60 µs read cadence makes stale reads structural).



The full loop: free-running ADC → newest-triple publish → age-gated snapshot at the trigger → render + relink inside the 120 µs lead → corrected pass N+1. The green arrow landing before the tail *is* the N → N+1 contract.

Key numbers (default configuration)

Quantity	Value	Source
Bitstream sample clock	20 MHz (50 ns/sample)	`SIGNAL_DMA_SAMPLE_RATE_HZ` , D2
Default cycle	60 µs (10/20/10/20 µs, modes 7/0/7/0)	`signal_init_default_dataset()`
Pass floor / default pass	≥ 200 µs → 4 copies = 240 µs	`DMA_MIN_PASS_SAMPLES` , `dma_compute_copies()`
Control lead	120 µs before tail	`DMA_CONTROL_LEAD_US` , D3/D4
Dead time	2 µs = 40 samples	`DEFAULT_DEAD_TIME_US`
ADC rate / triple pitch	250 kHz aggregate / 12 µs per triple	`g_analog_continuous_sample_hz`
ADC frame	16 triples = 192 µs; store 4 frames deep	`ADC_CONTINUOUS_FRAME_TRIPLES`

Age budget at control point	max(cycle window, 288 µs floor)	`analog_min_snapshot_age_us()`
Control auto-off threshold	3 consecutive misses	`signal_control_update_corrections()`

Source references

- `main/src/signal_engine_dma.cpp` — renderer, descriptor chains, ISR, control task loop
- `main/include/signal_engine_dma.h` — sample-clock definition (D2), render contract (D5, D10)
- `main/src/signal_controller.cpp` — `signal_control_update_corrections()`, age budget derivation, auto-off
- `main/src/helper_analog.cpp` — continuous pipeline, ISR timestamps, seqlock publish, `analog_read_control_snapshot()`, latency stopwatch
- `docs/adr/0001-dual-signal-engine-i2s-dma.md` — engine design decisions D1–D10